

Katlaz++

Official English Documentation

Complete manual for the Katlaz language and KatlazApp framework: syntax, routes, visual bridge, HTML View, Python integration, C/C++ integration, runtime workflow and best practices.

Table of Contents

1. Overview
2. Recommended Project Structure
3. Installation and Environment
4. Katlaz Language Concepts
5. Variables, Types and Expressions
6. Routes
7. Models and SQLite
8. Emit, Return and Frontend Events
9. File System API
10. Python Integration
11. C and C++ Integration
12. KatlazApp Visual Window and HTML View
13. Bridge: HTML/JavaScript to Katlaz
14. Native WebView Bridge
15. Full Example: Counter App
16. CLI Workflow
17. Internal Architecture
18. Error Handling
19. Security Best Practices
20. Quick Reference
21. Troubleshooting
22. Roadmap

1. Overview

Katlaz++ is the documentation package for the Katlaz language and the KatlazApp application framework.

Katlaz is the programming language layer. It uses `.katlaz`` files and is designed to transform Katlaz code into executable output, mainly Python or C, with a path toward native binaries.

KatlazApp is the application layer. It lets you build projects with a visual HTML interface, Katlaz routes in the backend, a bridge between the visual window and Katlaz code, SQLite storage, file-system commands, Python calls and C/C++ native calls through shared libraries.

Main goals:

- write backend logic in Katlaz;
- transpile/compile that logic to Python or C;
- run routes called from the visual window;
- use HTML/CSS/JavaScript as the View layer;
- connect the View to the backend through a bridge;
- extend the language with Python and C/C++ integrations.

2. Recommended Project Structure

Recommended structure for the revised repository:

```
katlaz-main/  
  README.md  
  INSTALL.md  
  KATLAZ_FIXES.md  
  katlaz/  
    README.md  
    parser.py  
    codegen.py  
    compiler.py  
    c_generator.py  
    lexer.py  
    transformer.py  
    type_checker.py  
    cli.py  
  katlazapp/  
    README.md  
    main.py  
    setup.py  
    build.bat  
    katlaz.c  
    katlaz.pyx
```

Conceptual separation:

- ``katlaz``: language, parser, AST, type checking, code generation and compiler.
- ``katlazapp``: application framework, runtime, CLI, visual server and bridge.

A typical application project generated by KatlazApp should look like this:

```
my_app/  
  main.katlaz  
  main.py          # generated backend  
  app/  
    app.html       # visual view  
    katlaz.js      # browser bridge helper  
  assets/
```

3. Installation and Environment

Minimum requirements:

- Python 3.10 or newer;
- pip;
- GCC, Clang, MinGW or Visual Studio Build Tools for native builds;
- optional GTK/WebKit dependencies for native desktop mode;
- a modern browser for the HTTP visual mode.

Install locally from the corrected project:

```
cd katlaz-main
python -m pip install -e .
```

For KatlazApp development:

```
cd katlazapp
python -m pip install -e .
```

Recommended test flow:

```
katlazapp create my_app
cd my_app
katlazapp build main.katlaz
katlazapp serve
```

Then open the visual window in the browser:

<http://localhost:3000>

4. Katlaz Language Concepts

Katlaz uses ``.katlaz`` files and indentation-based blocks. The current system supports two related styles:

1. Language/compiler style for native or C-oriented compilation.
2. KatlazApp DSL style for routes, views and the visual bridge.

Core concepts:

- ``route``: defines callable backend behavior.
- ``func``: defines a normal function in the language compiler layer.
- ``model``: defines a SQLite-oriented data structure in the app layer.
- ``emit``: sends an event/result to the frontend.
- ``return``: returns a value from a route or function.
- ``py.call``: calls a Python function.
- ``cpp.call``: calls a native C/C++ function exported from a shared library.

Minimal KatlazApp route:

```
route hello:
    emit "notify", "Hello from Katlaz"
```

Minimal typed route:

```
route greet(name:string):
    emit "notify", "Hello " + name
```

5. Variables, Types and Expressions

Katlaz supports basic scalar values and typed parameters. In the app runtime, accepted types include:

- ``int``
- ``float``

- `string` or `str`
- `bool`
- `any`

Examples:

```
route calc:
  x = 10
  y = x + 5
  emit "result", y
```

Compiler-oriented typed variables may use this style:

```
x:int = 10
y:float = 3.14
name:str = "Igor"
flag:bool = true
```

For functions:

```
func add(a:int, b:int) -> int:
  return a + b
```

6. Routes

Routes are callable backend actions. A route may receive values from the visual window and can emit events or return values.

Example route with parameters:

```
route sum(a:int, b:int):
  result = a + b
  emit "result", result
```

From JavaScript:

```
katlaz.call("sum", { a: 4, b: 8 })
```

Expected flow:

```
HTML button or JavaScript action
-> katlaz.call(name, data)
-> bridge receives JSON
-> runtime executes route
-> route emits/returns response
-> frontend receives event/result
```

7. Models and SQLite

KatlazApp can use simple model declarations to describe SQLite tables.

Example model:

```
model users:
  id int
  name string
```

Insert data:

```
route add_user(name:string):
  db.insert "users", {name: name}
  emit "notify", "User saved"
```

Select data:

```
route list_users:
  users = db.select "users"
  emit "users", users
```

Recommended database practices:

- validate route parameters before inserts;
- avoid storing raw untrusted data without validation;
- keep table names predictable;
- make backup/export routines before destructive operations.

8. Emit, Return and Frontend Events

`emit` sends an event to the frontend. The visual layer receives it as a browser event or as a normalized bridge response.

Katlaz:

```
route notify:
    emit "notify", "Saved successfully"
```

JavaScript:

```
window.addEventListener("notify", event => {
    console.log(event.detail)
})
```

Routes can also return raw values:

```
route get_number:
    return 123
```

Recommended response pattern:

- use `emit` for UI updates;
- use `return` for direct API-style values;
- keep error responses serializable as JSON.

9. File System API

The application runtime can expose file operations through the Katlaz DSL.

Write file:

```
route save_text(text:string):
    fs.write "output.txt", text
    emit "notify", "File saved"
```

Read file:

```
route load_text:
    data = fs.read "output.txt"
    emit "loaded", data
```

Security recommendations:

- do not allow arbitrary absolute paths from the UI;
- restrict file access to the project directory or a configured workspace;
- block path traversal such as `../`;
- never expose secrets through frontend events.

10. Python Integration

KatlazApp can call Python functions. This gives Katlaz access to the Python ecosystem for math, data processing, automation, AI tools, APIs and system scripts.

Basic example:

```
route square_root(x:int):
    py.call "math.sqrt" ${x}
```

`\${x}` means: pass the current Katlaz variable `x` to the Python call. Without `\${}`, the argument is treated as a literal value.

Example calling a custom module:

```
# tools.py
def greet(name):
    return "Hello " + name

route hello(name:string):
    py.call "tools.greet" ${name}
```

Recommended rules:

- keep Python functions pure when possible;
- return JSON-serializable values;
- handle Python exceptions and convert them into user-friendly Katlaz errors;
- never call unsafe Python modules from untrusted UI input.

11. C and C++ Integration

KatlazApp can call native functions through shared libraries. This is useful for performance-critical logic or integration with existing C/C++ code.

C++ function exported with C ABI:

```
extern "C" long long add(long long a, long long b) {
    return a + b;
}
```

Compile on Linux:

```
g++ -shared -fPIC demo.cpp -o libdemo.so
```

Compile on Windows with MinGW:

```
g++ -shared demo.cpp -o demo.dll
```

Call from KatlazApp:

```
route native_sum:
    cpp.call "./libdemo.so" "add" 2 3
```

Recommended rules:

- export functions with `extern "C"` to avoid C++ name mangling;
- use simple ABI-safe argument types first, such as integers and floats;
- keep memory ownership clear;
- validate library paths;
- do not load arbitrary libraries from user input.

12. KatlazApp Visual Window and HTML View

The visual window is the layer where KatlazApp runs the application interface. The recommended stable mode is the HTTP visual mode:

```
katlazapp serve
```

This serves the project at:

```
http://localhost:3000
```

Default visual file:

app/app.html

The visual window should contain:

- HTML for structure;
- CSS for design;
- JavaScript for UI events;
- `katlaz.js` or an equivalent bridge helper for backend calls.

Minimal visual interface:

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>KatlazApp</title>
</head>
<body>
  <h1>KatlazApp View</h1>
  <button onclick="katlaz.call('hello')">Run</button>
  <p id="status">Waiting...</p>

  <script src="katlaz.js"></script>
  <script>
    window.addEventListener('notify', event => {
      document.getElementById('status').innerText = event.detail
    })
  </script>
</body>
</html>
```

13. Bridge: HTML/JavaScript to Katlaz

The bridge connects the frontend View to the backend Katlaz route system.

Recommended JavaScript helper:

```
window.katlaz = {
  call: async function(name, data = {}) {
    const response = await fetch('/api', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, data })
    })
    const result = await response.json()
    if (result.events) {
      for (const ev of result.events) {
        window.dispatchEvent(new CustomEvent(ev.name, { detail: ev.data }))
      }
    }
    return result
  }
}
```

Katlaz route:

```
route hello(name:string):
  emit "notify", "Hello " + name
```

HTML button:

```
<button onclick="katlaz.call('hello', {name: 'Igor'})">
  Say hello
</button>
```

Expected request payload:

```
{
  "name": "hello",
  "data": {"name": "Igor"}
}
```

Expected response shape:

```
{
  "ok": true,
  "events": [{"name": "notify", "data": "Hello Igor"}]
}
```

14. Native WebView Bridge

Desktop mode may use a native WebView bridge instead of HTTP. In that case, JavaScript sends messages through the WebView host.

Conceptual JavaScript:

```
window.katlaz = {
  call: function(name, data = {}) {
    const payload = JSON.stringify({ name, data })
    window.webkit.messageHandlers.katlaz.postMessage(payload)
  }
}
```

Bridge responsibilities:

- accept `str`, `bytes` or object/dictionary payloads;
- normalize messages into `{name, data}`;
- call the runtime route dispatcher;
- return JSON-serializable responses;
- expose clear errors when route names or payloads are invalid.

If desktop mode fails, use `katlazapp serve` as the stable development path because it exercises the same backend logic through HTTP.

15. Full Example: Counter App

`main.katlaz`:

```
route increment(value:int):
  next = value + 1
  emit "counter", next
  return next
```

`app/app.html`:

```
<!doctype html>
<html>
<body>
  <h1>Counter</h1>
  <p id="count">0</p>
  <button id="btn">Increment</button>

  <script src="katlaz.js"></script>
  <script>
    let current = 0
    document.getElementById('btn').onclick = async () => {
      await katlaz.call('increment', { value: current })
    }
    window.addEventListener('counter', e => {
      current = e.detail
      document.getElementById('count').innerText = current
    })
  </script>
</body>
</html>
```

Run:

```
katlazapp build main.katlaz
katlazapp serve
```


16. CLI Workflow

Common KatlazApp commands:

```
katlazapp create my_app
katlazapp build main.katlaz
katlazapp serve
katlazapp desktop
```

Typical workflow:

1. Edit main.katlaz
2. Edit app/app.html
3. Build the Katlaz backend
4. Start the visual server
5. Test UI calls
6. Debug bridge responses

For the language compiler layer:

```
katlaz hello.katlaz -o hello
```

Then run the generated binary:

```
./hello
```

or on Windows:

```
hello.exe
```

17. Internal Architecture

Katlaz compiler pipeline:

```
.katlaz source
-> lexer/parser
-> AST
-> analyzer/type checker
-> code generator
-> C/Python output
-> compiler/runtime
```

KatlazApp runtime pipeline:

```
HTML View
-> katlaz.call()
-> HTTP or WebView bridge
-> route dispatcher
-> generated Python runtime
-> emit/return
-> frontend event/result
```

Important modules:

- parser: reads Katlaz syntax and builds structures.
- transpiler/codegen: converts Katlaz to Python or C.
- runtime/core: route registry and execution.
- runtime/bridge: message normalization between UI and backend.
- runtime/app: visual HTTP server and static file serving.
- runtime/native: Python and C/C++ extension calls.

18. Error Handling

Errors should be readable and serializable.

Syntax error example:

```
[Katlaz Syntax Error] line 3: expected ':' after route declaration
```

Type error example:

```
{
  "ok": false,
  "code": "KATLAZ_TYPE_ERROR",
  "error": "Invalid type for 'name'",
  "hint": "Expected string"
}
```

Runtime error example:

```
{
  "ok": false,
  "code": "KATLAZ_RUNTIME_ERROR",
  "error": "division by zero"
}
```

Bridge error example:

```
{
  "ok": false,
  "code": "KATLAZ_BRIDGE_ERROR",
  "error": "Route not found: hello2"
}
```

Good error messages should include:

- the error code;
- the route or function name when safe;
- a human-readable message;
- a hint for the developer.

19. Security Best Practices

Recommended security rules for KatlazApp projects:

- validate every route parameter;
- never pass raw user input directly into Python or C++ calls;
- restrict file-system access to a known workspace;
- do not expose secrets in frontend events;
- do not allow the browser to choose arbitrary native library paths;
- sanitize database inputs;
- add authentication before exposing KatlazApp on a public network;
- prefer localhost-only development servers by default.

The visual window is powerful because it can call backend routes. Treat it like a local app API and protect dangerous operations.

20. Quick Reference

Routes:

```
route hello:
  emit "notify", "Hello"
```

Typed route:

```
route greet(name:string):
  emit "notify", name
```

Return:

```
route value:
```

```
return 42
```

Python call:

```
route calc(x:int):  
  py.call "math.sqrt" ${x}
```

C/C++ call:

```
route native_sum:  
  cpp.call "./libdemo.so" "add" 2 3
```

Frontend call:

```
katlaz.call('hello')  
katlaz.call('greet', {name: 'Igor'})
```

Serve visual app:

```
katlazapp serve
```

Build app:

```
katlazapp build main.katlaz
```

21. Troubleshooting

Problem: the visual window opens but buttons do nothing.

Checklist:

- confirm `katlaz.js` is loaded;
- open browser DevTools and check console errors;
- confirm `/api` is reachable;
- confirm route names match exactly;
- rebuild `main.katlaz` after edits.

Problem: route not found.

Checklist:

- check route spelling;
- rebuild the app;
- restart the server;
- inspect the generated Python route registry.

Problem: Python integration fails.

Checklist:

- confirm the Python module is importable;
- confirm the function name is correct;
- return JSON-serializable values;
- check stack traces in the server terminal.

Problem: C/C++ integration fails.

Checklist:

- export with `extern "C"`;
- verify the shared library path;

- verify argument types;
- use `ldd` on Linux or dependency tools on Windows;
- test the shared library independently before calling it from Katlaz.

22. Roadmap

Recommended next improvements for the Katlaz ecosystem:

- more complete grammar coverage;
- official package manager;
- stable standard library;
- stronger type inference;
- better IDE syntax highlighting;
- hot reload for KatlazApp;
- WebSocket bridge as a first-class mode;
- authentication and middleware;
- native desktop packaging;
- LLVM or WebAssembly backend;
- automated test suite for parser, transpiler, bridge and runtime.

Katlaz++ should evolve toward a unified developer experience: simple syntax, strong bridge, native extensions and fast visual app creation.